PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

DESARROLLO FRONTEND CON REACT, REACT ROUTER DOM Y CONSUMO DE API REST

Presentado a:	Alexandra Guevara
Por Aprendiz:	**luis fernando conejo**
Ficha:	3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 8 del 11 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

FASE 1: Preparación del proyecto frontend e investigación teórica

-mi-proyecto/
backend/
-server.js => Servidor Express, conexión a MySQL y rutas REST
* package.json
* frontend/
* public/
* src/
-components/
* pages/
*Login.jsx
*Registro.jsx
*Dashboard.jsx
*Productos.jsx
*Contacto.jsx
*App.jsx => Configuración principal de React Router DOM (Routes y Route)
* main.jsx
*index.css

* ¿Qué es React y qué función cumple en el frontend? Es una librería para crear páginas web interactivas por medio de piezas separadas (componentes). En la parte visual (frontend), sirve para actualizar la pantalla al instante sin tener que recargar todo el navegador cada vez que el usuario hace un cambio.

* ¿Qué es React Router DOM? Es una herramienta que sirve para simular que cambias de página web dentro de una misma aplicación, permitiendo navegar entre el login, el dashboard y los productos sin que la página se quede en blanco o se recargue por completo.

*¿Cuál es la diferencia entre Link y useNavigate()?

*El Link es simplemente un enlace visual (como un botón o etiqueta <a>) donde el usuario hace clic para ir a otra vista.

*El useNavigate() es una orden de código (una función) que se ejecuta por dentro, por ejemplo, para mandar al usuario a otra página automáticamente después de que guardó un producto o dio clic en ingresar.

* ¿Qué función cumplen BrowserRouter, Routes y Route?

*BrowserRouter: Es el contenedor principal que le avisa a React que vamos a usar rutas en el navegador.

*Routes: Agrupa y revisa todas las rutas que tenemos creadas.

*Route: Es cada ruta individual que conecta una dirección de internet específica con su respectiva pantalla (componente).

* ¿Qué es useState y cómo permite controlar los campos de un formulario? Es una memoria temporal (estado) que guarda lo que el usuario va escribiendo. Gracias a esto, cada vez que tecleas en un input, el sistema atrapa ese valor al instante y lo deja listo para enviarlo.

*¿Qué función cumple fetch() al consumir una API REST? Es la función de JavaScript que sirve para comunicarse con el servidor (backend). Se encarga de hacer las peticiones para pedir datos (GET) o para enviarlos (POST).

*¿Qué relación existe entre el frontend, el backend y la base de datos?

*El frontend (React) es lo que ve el usuario en la pantalla.

*El backend (Node.js/Express) es el que procesa la información y recibe las órdenes.

*La base de datos (MySQL) es donde se guarda todo de forma permanente para que no se borre. El frontend le habla al backend, y el backend guarda o busca los datos en la base de datos.

FASE 2: Implementación de la navegabilidad

Cómo se estructuró la navegación:

*Configuración de rutas: En el archivo principal (App.jsx), se utilizó BrowserRouter, Routes y Route para asociar cada URL con su respectivo componente sin recargar la página.

*Pantallas principales:

*/ (Login): Pantalla de práctica. Cuenta con un botón de ingreso que utiliza useNavigate() para redirigir directamente al Dashboard sin validar credenciales.

*/registro (Registro): Accesible desde el login mediante un componente Link.

*/dashboard (Dashboard): Menú principal que sirve como centro de control para saltar a Productos y Contacto.

*/productos (Productos): Pantalla de gestión de inventario.

*/contacto (Contacto): Información básica de la empresa.

*Retorno: Cada página secundaria incluye botones con funciones tradicionales para regresar al Dashboard o cerrar sesión y volver al Login.

FASE 3: Construcción del formulario de productos

Funcionamiento del formulario:

*Control de datos con useState: Cada campo (nombre, categoria, precio, cantidad) tiene su propio estado inicializado en vacío (""), el cual se actualiza en tiempo real cada vez que el usuario escribe en los inputs.

*Validación de tipos: Al momento de empaquetar el objeto para enviarlo, los campos de precio y cantidad se convierten explícitamente a números usando Number() para evitar errores en la base de datos.

*Petición HTTP POST:

*Se utiliza fetch("http://localhost:5000/productos", { method: "POST", ... }).

*Se configuran los headers con "Content-Type": "application/json" para avisar al backend que se le enviará un formato estructurado.

*Se utiliza JSON.stringify() para transformar el objeto de JavaScript en un texto plano con formato JSON que pueda viajar sin problemas por la red.

*Respuesta y limpieza: Si el servidor responde correctamente (respuesta.ok), se muestra una alerta de éxito, se limpian los inputs dejándolos en blanco y se llama automáticamente a la función de consulta para actualizar la tabla al instante.

FASE 4: Consulta de productos desde el backend

Para mostrar en la tabla los datos reales que están guardados en la base de datos de MySQL:

*Petición GET: Al hacer clic en el botón de consultar, se ejecuta un fetch() hacia la ruta GET /productos del backend.

*Procesamiento de datos: La respuesta del servidor se convierte en un formato legible mediante .json().

*Estado de la lista: Esos datos se guardan en un estado con corchetes (useState([])) para que React sepa que es una lista de elementos.

*Pintado en la tabla: Mediante el método .map(), se recorre la lista producto por producto para dibujar de forma dinámica las filas (<tr>) y celdas (<td>) con el ID, nombre, categoría, precio y cantidad reales (cero datos de ejemplo o inventados).

*Actualización automática: Cada vez que se registra un producto nuevo con éxito, la función de consulta se vuelve a ejecutar sola para que la tabla se refresque al instante.

FASE 5: Integración entre frontend y backend

*Servidor en marcha: El backend desarrollado en Node.js con Express se ejecuta en el puerto 5000 y está configurado para entender y responder solicitudes en formato JSON (express.json()).

*Comunicación sin bloqueos (CORS): Como React corre en un puerto diferente (por ejemplo, en Vite), se utiliza el middleware cors en Express para permitir que el navegador acepte las peticiones entre ambos entornos sin restricciones de seguridad.

*Persistencia en MySQL: Las peticiones POST guardan la información directamente en la tabla producto de la base de datos relacional. Si se apaga o se reinicia el servidor, los datos no se pierden; siguen ahí disponibles al hacer una nueva consulta GET.

FASE 6: Revisión y análisis de errores (8 errores comunes)

Ruta 404 en el fetch

*¿Cuál es el error? La URL configurada en el fetch no coincide con la ruta definida en Express (ej: /api/productos en vez de /productos).

*Por qué ocurre: Desalineación entre las rutas del servidor y las peticiones del frontend.

*Consecuencia: El servidor rechaza la petición indicando que el recurso no existe.

*Cómo corregirlo: Verificar y estandarizar la ruta exacta en ambos extremos.

*Prueba: Hacer la petición y revisar en la pestaña Network del navegador que devuelva código 200.

Falta del encabezado Content-Type

*¿Cuál es el error? Enviar una petición POST sin los headers de tipo de contenido.

*Por qué ocurre: Omitir la configuración de las cabeceras en el objeto de configuración del fetch.

*Consecuencia: El servidor no sabe cómo interpretar el cuerpo y el req.body llega vacío o como undefined.

*Cómo corregirlo: Agregar "Content-Type": "application/json" en los headers de la petición.

*Prueba: Comprobar en el backend que req.body imprime correctamente el objeto recibido.

Ausencia de JSON.stringify()

*¿Cuál es el error? Enviar un objeto de JavaScript nativo directamente en el body del fetch.

*Por qué ocurre: Intentar mandar el objeto sin convertirlo a texto plano.

*Consecuencia: Los datos viajan corruptos o como texto plano inválido ([object Object]).

*Cómo corregirlo: Envolver siempre el objeto a enviar dentro de JSON.stringify().

*Prueba: Verificar que el cuerpo de la petición HTTP en la red muestre un JSON válido en formato texto.

Bloqueo por CORS

*¿Cuál es el error? El navegador bloquea la comunicación entre el frontend y el backend.

*Por qué ocurre: Frontend y backend corren en puertos diferentes y el servidor no tiene activo el middleware de CORS.

*Consecuencia: La consola del navegador muestra un error rojo de seguridad por políticas de origen cruzado.

*Cómo corregirlo: Instalar y configurar app.use(cors()) en el archivo principal de Express.

*Prueba: Consumir la API desde React sin que la consola arroje errores de bloqueo.

Uso incorrecto del estado inicial (useState("") vs useState([]))

*¿Cuál es el error? Inicializar la lista de productos con comillas vacías en lugar de un arreglo.

*Por qué ocurre: Confundir un estado de texto con un estado que debe almacenar múltiples registros.

*Consecuencia: Al intentar recorrer los datos con .map() para pintar la tabla, la aplicación se cae con un error de ejecución.

*Cómo corregirlo: Inicializar el estado de la lista con corchetes vacíos: useState([]).

*Prueba: Cargar la página de productos y verificar que la tabla dibuje los elementos sin crashearse.

Inputs que no limpian su valor tras registrar

*¿Cuál es el error? Los campos del formulario conservan la información escrita después de un registro exitoso.

*Por qué ocurre: No invocar las funciones despachadoras del estado (ej: setNombre("")) al completarse el POST.

*Consecuencia: El usuario ve los datos viejos y puede reenviarlos por error.

*Cómo corregirlo: Limpiar cada estado individual dentro del bloque if (respuesta.ok).

*Prueba: Registrar un producto con éxito y comprobar que los inputs queden en blanco automáticamente.

Caída de conexión con el backend

*¿Cuál es el error? Intentar hacer peticiones cuando el servidor de Node.js está apagado.

*Por qué ocurre: Olvidar ejecutar node server.js o que el puerto esté ocupado.

*Consecuencia: Las peticiones fallan por tiempo de espera y la interfaz no trae ni guarda nada.

*Cómo corregirlo: Mantener la terminal del backend activa y validando su ejecución.

*Prueba: Apagar el servidor y comprobar que la interfaz maneje el fallo sin romper la aplicación.

Confusión de tipos de datos al enviar a MySQL

*¿Cuál es el error? Enviar precios o cantidades como texto plano en lugar de valores numéricos.

*Por qué ocurre: Los inputs de HTML devuelven cadenas (string) por defecto.

*Consecuencia: Posibles rechazos o errores de inserción en columnas de tipo entero o decimal en la base de datos.

*Cómo corregirlo: Envolver los valores con Number() antes de armar el objeto a enviar.

*Prueba: Registrar un producto e inspeccionar directamente en la base de datos que los campos numéricos se guarden con su tipo correcto.

FASE 7: Pruebas funcionales

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

*Abrir Login: El sistema muestra el formulario de práctica correctamente.



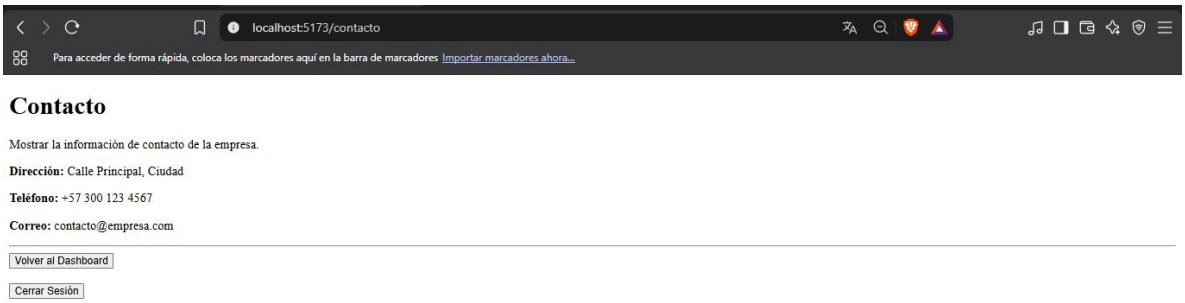
---

*Ir de Login a Dashboard: Al presionar el botón de ingreso, redirige de forma exitosa usando useNavigate().



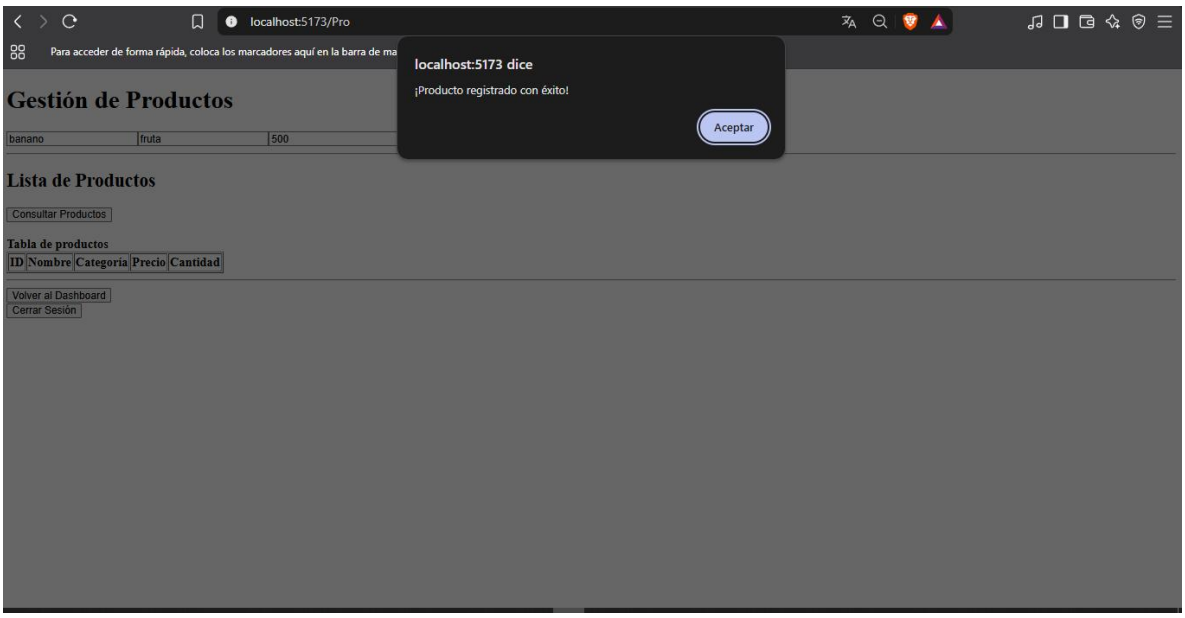
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

*Navegar por las vistas: Es posible saltar entre Productos, Contacto y el Dashboard sin que la página se recargue por completo.



---

*Registrar un producto válido: Los datos ingresados viajan por POST, se guardan en MySQL y se muestra la alerta de éxito.



*Consultar productos: Al hacer clic en consultar, los registros de la base de datos se pintan de forma dinámica en la tabla mediante el .map().



. Preguntas de análisis (Directo al grano)

¿Qué diferencia hay entre una ruta del frontend y una del backend?

*La del frontend es la dirección visual que te cambia en la barra del navegador (como /productos) para mostrarte otra pantalla sin recargar la página.

*La del backend es el punto de acceso (endpoint) del servidor (como /productos por GET o POST) que recibe la orden, hace la lógica y va a la base de datos.

¿Para qué sirve React Router DOM?

* Es la herramienta que montamos para simular que cambiamos de página web dentro de React, sin que el navegador se quede en blanco o se recargue completo.

¿Cuál es la diferencia entre Link y useNavigate()?

*El Link es un enlace visual (un botón o etiqueta) donde el usuario hace clic para ir a otra vista.

*El useNavigate() es una orden de código que se dispara sola (por ejemplo, para mandarte al Dashboard automáticamente cuando le das clic a ingresar o guardas un producto).

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

¿Por qué el frontend no se debe conectar directo a MySQL?

* Por pura seguridad. Si pones las credenciales de la base de datos en el navegador, cualquiera las puede ver. De eso se encarga el servidor (backend) por debajo.

¿Qué hace el fetch()?

* Es la función de JavaScript para hablarle al servidor: sirve tanto para pedirle datos (GET) como para mandárselos (POST).

¿Por qué usamos JSON.stringify() en un POST?

* Porque la red y los servidores no entienden objetos de JavaScript crudos. Esa función aplana tu objeto y lo vuelve un texto plano en formato JSON para que viaje bien.

¿Qué pasa si la URL del backend está mal escrita?

* El servidor nunca recibe nada y el navegador te bota error o un 404 Not Found, dejando la app sin traer ni guardar nada.

¿Por qué sale el error de CORS?

* Porque el navegador es jodido con la seguridad: si el frontend y el backend corren en puertos diferentes, el navegador bloquea la comunicación a menos que le pongas el permiso (cors()) en el servidor.

¿Cómo se actualiza la tabla al registrar un producto

*Apenas el servidor dice que guardó bien, llamamos de una vez a la función que hace el fetch (GET) para traer los datos frescos de MySQL y meterlos en el estado que dibuja la tabla.

¿Por qué hay que validar los datos en el frontend y en el backend?

*En el frontend para avisarle rápido al usuario si metió mal un dato.

*En el backend por seguridad seria, porque alguien pilo se puede saltar el navegador y mandar datos corruptos directo al servidor.

¿Cómo compruebas que un producto sí quedó guardado en MySQL?

* Mirando directo en la base de datos con una herramienta (como MySQL Workbench) o tirando un SELECT * FROM producto; en la consola para ver que el registro físico sí exista.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

¿Qué diferencia hay entre navegar a una página y proteger una ruta?

*Navegar: Es moverse libremente de una pantalla a otra.

*Proteger una ruta: Es ponerle un candado o condición (como revisar si inició sesión); si no cumple, se devuelve al usuario al Login sin dejarlo pasar.